Ulster University

COM161 – Problem Solving for Computing

UG Computing

AY: 2023/24

Lecture & Tutorial

Week 7

Attendance Monitoring

Quickly

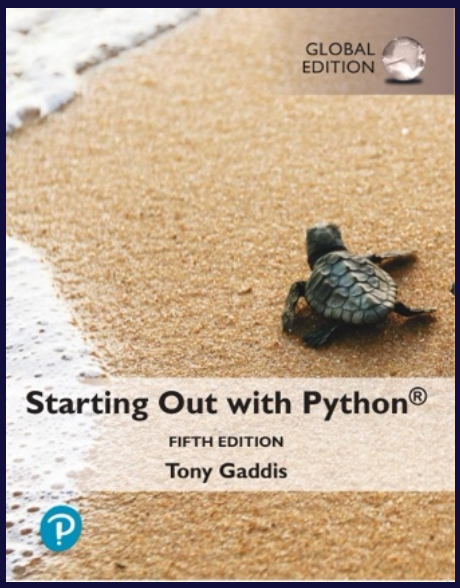
Pearson Copyright © 2018 Pearson Education, Ltd.

Outline

Week 7 (Lecture & Tutorial)

- **Section 1: Working with Lists**
  - Creating, populating and manipulating
- **Section 2: Brief overview of Tuples**
  - Focusing on core differences to Lists

Pearson Copyright © 2018 Pearson Education, Ltd.



Working with Lists

Section

Sequences

- A sequence is a generic term for an object that points to multiple items of data.
  - items are stored in *sequence* one after another
  - possible to perform operations on a sequence to examine and manipulate the items stored in it.

- Python provides different types of sequences, including **lists** and **tuples**
  - The simple difference between these is that a **list** is *mutable* while a **tuple** is *immutable*

  - As such, we will focus on list structures as a means to introduce these sequences.

Pearson Copyright © 2022 Pearson Education, Ltd. All Rights Reserved.

Introduction to Lists (1)

- Format: `list = [item1, item2, etc]`
  - *Lists are **dynamic** so you can add and remove items from them*
  - *Lists can also hold an assortment of items of **different types***

- An **element** refers to an item in a list
  - `even_num = [2, 4, 6, 8, 10]`

Figure 7-2 A list of strings

Figure 7-1 A list of integers

Figure 7-3 A list holding different types

Pearson Copyright © 2022 Pearson Education, Ltd. All Rights Reserved.

Introduction to Lists (2)

- Lists provides a range of methods similar to **strings**
  - *e.g. use the `len()` function and square brackets `[]` to access data, with the first element at index 0*

- `print` function can be used to display an entire list
  - E.g. `print(even_num)`

- The `list()` function can convert certain types of objects to lists

```
numbers = list(range(5))
```

Pearson Copyright © 2022 Pearson Education, Ltd. All Rights Reserved.

Traversing a List

- The most common way to traverse the elements of a list is with a for loop. The syntax is the same as for strings:

```
for cheese in cheeses:
print(cheese)
```

- This works well if you only need to read the elements of the list.

- But what if you want to write or update the elements?
  - For that, you will need indexing.

Ulster University

Pearson Copyright © 2022 Pearson Education, Ltd. All Rights Reserved.

List Indexing (1)

- A common way to index a list is to combine the built-in functions `range` and `len`:

```
for i in range(len(numbers)):
numbers[i] = numbers[i] * 2
```

- This loop traverses the list and updates each element.
  - `len` returns the number of elements in the list.
  - `range` returns a list of indices from 0 to n □ 1, where n is the length of the list.

- Each time through the loop `i` gets the index of the next element.
- The assignment statement in the body uses i to read the old value of the element and to assign the new value.

Ulster University

Pearson Copyright © 2022 Pearson Education, Ltd. All Rights Reserved.

List Indexing (2)

Points to note

- A `for` loop over an empty list never runs the body:

```
for x in []:
print('This never happens.')
```

- Although a list can contain another list, the nested list still counts as a single element in the outer list.

- E.g. The length of this list is four:
  `['spam', 1, ['Brie', 'Roquefort', 'Pol le Veq'], [1, 2, 3]]`

Ulster University

Pearson Copyright © 2022 Pearson Education, Ltd. All Rights Reserved.

Concatenating Lists

- Similar to String objects, the **+ operator** can be used to concatenate two lists.
- The **+= augmented assignment operator** can also be used to concatenate lists

```
list1 = [1,2,3,4]
list2 = [3,4,5]
new_list = list1+list2
print(new_list)
new_list+=[4,4,4]
print(new_list)

numbers = list(range(3))
print(numbers)

numbers = [1, 2, 3, 4, 5]
print(numbers[-2])

days = ['Sun','Mon','Tues', 'Wed', 'Thur', 'Fri', 'Sat']
mid_days = days[len(days)//2]
print(mid_days)
```

```
# OUTPUT
[1, 2, 3, 4, 3, 4, 5]
[1, 2, 3, 4, 3, 4, 5, 4, 4, 4]
[0, 1, 2]
4
['Wed']
```

Ulster University

Pearson Copyright © 2022 Pearson Education, Ltd. All Rights Reserved.

Repetition Operator

- The **repetition operator** makes multiple copies of a list and joins them together
  - The * symbol acts as a repetition operator when applied to a **sequence** and an **integer**
    - *Sequence is left operand, integer is right operand, in the following format:*
    - *list * n*
  - where n is the number of copies to make

```
>>> numbers = [1,2,3]  * 3
>>> print[numbers]
[1, 2, 3, 1, 2, 3, 1, 2, 3]
```

Ulster University

Pearson Copyright © 2022 Pearson Education, Ltd. All Rights Reserved.

List Example 1

Assigning User Input to a List

```
NUM_DAYS = 5

def main():
    # Create a list to hold the sales for each day.
    sales = [0] * NUM_DAYS

    # Create a variable to hold an index.
    index = 0
    print('Enter the sales for each day.')

    # Get the sales for each day.
    while index < NUM_DAYS:
        print('Day #', index + 1, ': ', sep='', end='')
        sales[index] = float(input())
        index += 1

    # Display the values entered.
    print('Here are the values you entered:')
    for value in sales:
        print('£',format(value,'.2f'),sep='')

# Call the main function.
main()
```

```
Enter the sales for each day.
Day #1: 340
Day #2: 360
Day #3: 400
Day #4: 280
Day #5: 120
Here are the values you entered:
£340.00
£360.00
£400.00
£280.00
£120.00
```

Ulster University

Pearson Copyright © 2022 Pearson Education, Ltd. All Rights Reserved.

List Slicing

- As with strings, it's possible to **slice** a list
  - List slicing format: *list[start : end]*

```
            0       1       2       3       4       5       6

>>>days = ['Mon', 'Tues', 'Wed', 'Thur', 'Fri', 'Sat', 'Sun']
>>>mid_days = days[2:3]
['Wed']
```

Note* The span includes a list containing copies of indexed elements from *start* up to, but **not including**, *end*

- *If start not specified, 0 is used for start index*
- *If end not specified, len(list) is used for end index*

- Slicing expressions can include a step value and negative indexes relative to end of list

Ulster University

Pearson Copyright © 2022 Pearson Education, Ltd. All Rights Reserved.

List Example 2

Interrogating a list

```
# Course List
courses = ['History', 'Math', 'Physics', 'CompSci']

# Print courses, length and individual elements
print(courses)
print(len(courses))
print(courses[0])

# Can also print out the last item using negative int.
print(courses[-1])

# print out the values up to but not including the last index
print(courses[0:2])

# OR if starting at beginning, can ignore 0
print(courses[:2])

# or just print every other course
print('Every other course:',courses[::2])
```

Ulster University

Pearson Copyright © 2022 Pearson Education, Ltd. All Rights Reserved.

Finding Items in Lists with the `in` Operator

- You can use the **`in`** operator to determine whether an item is contained in a list
  - General format: *`item in list`*
  - Returns `True` if the item is in the list, or `False` if it is not in the list

```
# Find if a value in a list e.g. return True or False
print('History' in courses)
print(French not in courses)
```

Ulster University

Pearson Copyright © 2022 Pearson Education, Ltd. All Rights Reserved.

Try it out

Exercise 1

```
# This program demonstrates the in operator used with a list, by
searching a list for product ID.

def main():
    # Create a list of product numbers.
    prod_nums = ['V475', 'F987', 'Q143', 'R688']

    # Q1. Get a product number from the keyboard, to search for.


    # Q2. Determine whether the product number is in the list.


    # Q3. Print out whether or not the product was found.


# Call the main function.
main()
```

Ulster University

Pearson Copyright © 2022 Pearson Education, Ltd. All Rights Reserved.

List Methods and Useful Built-in Functions

- **`append(item)`**: used to add items to a list
  - *`item`* is appended to the end of the existing list

- **`index(item)`**: used to return the location of an item in a list
  - Returns the index of the first element in the list containing `item`
    - *Raises `ValueError` exception if `item` not in the list*

```
# Adding an item
courses.append('Art')
print(courses) # Print an index to specific item;
print(courses.index('Art'))
```

Ulster University

Pearson Copyright © 2022 Pearson Education, Ltd. All Rights Reserved.

List Methods and Useful Built-in Functions (cont'd.)

- **`insert(index, item)`**: used to insert *`item`* at position *`index`* in the list

- **`sort()`**: used to sort the elements of the list in ascending order

- **`remove(item)`**: removes the **first** occurrence of *`item`* in the list

- **`reverse()`**: reverses the order of the elements in the list

Ulster University

Pearson Copyright © 2022 Pearson Education, Ltd. All Rights Reserved.

List Methods and Useful Built-in Functions (cont'd.)

- **`del` statement**: removes an element from a specific index in a list
  - General format: `del list[i]`

- **`min` and `max` functions**: built-in functions that returns the item that has the lowest or highest value in a sequence
  - The sequence is passed as an argument

Pearson Copyright © 2022 Pearson Education, Ltd. All Rights Reserved.

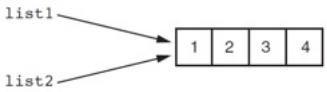
```python
# Adding an item to specific place; e.g. position 1
courses.insert(1,'German')
print(courses)

# Print an index to specific item;
print(courses.index('Art'))

# Adding several items using extend
courses_2 = ['French', 'Spanish']

courses.extend(courses_2)
print(courses)

# Removing specific named item
courses.remove('French')
print(courses)

# Removing last element
value = courses.pop()
print(courses)
print('Item popped was', value)

# Reverse list
courses.reverse()
print(courses)

# Sort list
courses.sort()
print(courses)

# Sort list of numbers
nums = [1, 5, 3, 7, 2]
nums.sort(reverse=True) # in this case, reversed
print(nums)

# Sort list of numbers without altering list
aSorted_nums = sorted(nums) # In this case, ascending
print(aSorted_nums)

# Min, Max and Sum function
print(min(nums), max(nums), sum(nums))
```

```
['History', 'German', 'Math', 'Physics', 'CompSci', 'Art']
5
['History', 'German', 'Math', 'Physics', 'CompSci', 'Art', 'French', 'Spanish']
['History', 'German', 'Math', 'Physics', 'CompSci', 'Art', 'Spanish']
['History', 'German', 'Math', 'Physics', 'CompSci', 'Art']
Item popped was Spanish
['Art', 'CompSci', 'Physics', 'Math', 'German', 'History']
['Art', 'CompSci', 'German', 'History', 'Math', 'Physics']
[7, 5, 3, 2, 1]
[1, 2, 3, 5, 7]
1 7 18
```

Copying Lists (1)

- Assignment with an = on lists **does not** make a copy.
- Instead, assignment makes the two variables **point** to the one list in memory.

Figure 7-4 `list1` and `list2` reference the same list



- This can cause issues - what might these be?

- To make a copy of a list you **must copy** each element of the list

Ulster University

Pearson Copyright © 2022 Pearson Education, Ltd. All Rights Reserved.

Copying Lists (2)

Method 1

- Creating a new **empty** list and use a `for` **loop** to add a copy of each element from the original list to the new list, using `append()`

```
# copy a list using a for loop
temp_courses = []
for each_course in courses:
    temp_courses.append(each_course)
print(temp_courses)
```

Ulster University

Pearson Copyright © 2022 Pearson Education, Ltd. All Rights Reserved.

Copying Lists (3)

Method 2

- Create a new empty list and **concatenate** the old list to the new empty list

```
# copy a list using concatenate
temp_courses = [] + courses
print(temp_courses)
```

Pearson Copyright © 2022 Pearson Education, Ltd. All Rights Reserved.

Example

Append

```
>>> t1 = [1, 2]
>>> t2 = t1.append(3)
>>> t1
[1, 2, 3]
>>> t2
None
```

- The return value from append is None.

+ Operator

```
>>> t3 = t1 + [4]
>>> t1
[1, 2, 3]
>>> t3
[1, 2, 3, 4]
```

- The result of the operator is a new list, and the original list is unchanged.

Pearson Copyright © 2022 Pearson Education, Ltd. All Rights Reserved.

Processing Lists

- List elements are often used in support of calculations, e.g.
  - To calculate **total** of numeric values in a list use loop with accumulator variable
  - To **average** numeric values in a list:
    - *Calculate total of the values*
    - *Divide total of the values by* `len(list)`
- List can be **passed as an argument** to a function

Ulster University

Pearson Copyright © 2022 Pearson Education, Ltd. All Rights Reserved.

List Example 3

Counting List Items

```python
# This program calculates the gross pay for each employee.

# NUM_EMPLOYEES is used as a constant for the size of the list.
NUM_EMPLOYEES = 6

def main():
    # Create a list to hold employee hours.
    hours = [0] * NUM_EMPLOYEES

    # Get each employee's hours worked.
    for index in range(NUM_EMPLOYEES):
        print('Enter the hours worked by employee ', index + 1, ': ', sep='', end='')
        hours[index] = float(input())

    # Get the hourly pay rate.
    pay_rate = float(input('Enter the hourly pay rate: '))

    # Display each employee's gross pay.
    for index in range(NUM_EMPLOYEES):
        gross_pay = hours[index] * pay_rate
        print('Gross pay for employee ', index + 1, ': £', \
              format(gross_pay, ',.2f'), sep='')

# Call the main function.
main()
```

```
Enter the hours worked by employee 1: 23
Enter the hours worked by employee 2: 32
Enter the hours worked by employee 3: 40
Enter the hours worked by employee 4: 12
Enter the hours worked by employee 5: 23
Enter the hours worked by employee 6: 23
Enter the hourly pay rate: 9.28
Gross pay for employee 1: £213.44
Gross pay for employee 2: £296.96
Gross pay for employee 3: £371.20
Gross pay for employee 4: £111.36
Gross pay for employee 5: £213.44
Gross pay for employee 6: £213.44
```

Ulster University

Pearson Copyright © 2022 Pearson Education, Ltd. All Rights Reserved.

Try it out

Exercise 2

```
# Extend the program to calculate the average gross pay?

    # Q1. Tally pay amounts

    # Q2. Calculate average (Tally / number of employee)

    # Q3. Print formatted average salary amount
```

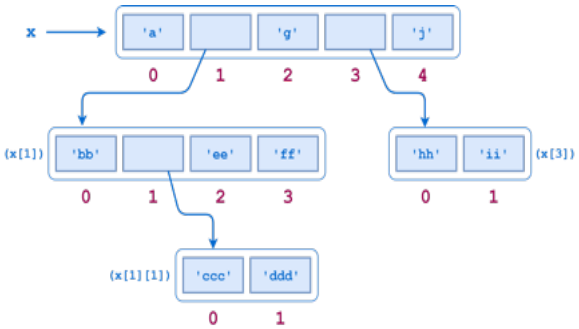
Pearson Copyright © 2022 Pearson Education, Ltd. All Rights Reserved.

Lists of Lists

- You have seen that an element in a list can be any sort of object.
  - that includes another list!

- A list can contain sublists, which in turn can contain sublists themselves, and so on to arbitrary depth, e.g.

```
>>> x = ['a', ['bb', ['ccc', 'ddd'], 'ee', 'ff'], 'g', ['hh', 'ii'], 'j']
>>> x

['a', ['bb', ['ccc', 'ddd'], 'ee', 'ff'], 'g', ['hh', 'ii'], 'j']
```



source: https://realpython.com/python-lists-tuples/

Pearson Copyright © 2022 Pearson Education, Ltd. All Rights Reserved.

Two-Dimensional Lists

- A **2D list** is a list that contains lists as its elements
  - Also known as **nested list**
  - Common to think of two-dimensional lists as having rows and columns
  - Useful for working with multiple sets of data

	Column 0	Column 1
Row 0	'Joe'	'Kim'
Row 1	'Sam'	'Sue'
Row 2	'Kelly'	'Chris'

- To process data in a two or more-dimensional list, we need to use two or more indexes
  - Typically use nested loops to process

	Column 0	Column 1	Column 2
Row 0	scores[0][0]	scores[0][1]	scores[0][2]
Row 1	scores[1][0]	scores[1][1]	scores[1][2]
Row 2	scores[2][0]	scores[2][1]	scores[2][2]

Ulster University

Pearson Copyright © 2022 Pearson Education, Ltd. All Rights Reserved.

Accessing Two-Dimensional Lists

Simple Examples

```
# what might this code do?
a = [[1, 2, 3], [4, 5, 6]]
print(a[0])
print(a[1])

a = [[1, 2, 3, 4], [5, 6], [7, 8, 9]]
for i in range(len(a)):
    for j in range(len(a[i])):
        print(a[i][j], end=' ')
    print()

a = [[1, 2, 3, 4], [5, 6], [7, 8, 9]]

for row in a:
    s = 0
    for elem in row:
        s += elem
print(s)
```

Ulster University

Pearson Copyright © 2022 Pearson Education, Ltd. All Rights Reserved.

List Example 4 (1)

Multi-dimensional Processing – Movies Example

- A file stores records, including *student id*, *student names* and their *marks*, as shown below (assume the maximum mark is 100).
  - Need a function `print_summary` of marks
  - Need a function `average` to print out average mark
  - Need a function `minimum` to print out lowest mark with associated actor details

```
# Actors Grade Example
B001, James Bond, 45
B002, Daniel Craig, 30
B007, Dr Who, 70.9
B008, Jonny English, 60.2
B009, Ethan Hunt, 20
```

Pearson Copyright © 2022 Pearson Education, Ltd. All Rights Reserved.

List Example 4 (2)

Reading the file – recap

```
# read in the data file, ignoring the '# comments' and append to a list
def read_file():
    infile = open("actor_grades.txt")

    # manual method for searching each string for commas
    for row in infile:
        start = 0
        string_builder=[]  # list to hold each row
        if not row.startswith("#"):  # ignoring the header comment in the data file
            for index in range(len(row)):
                 if row[index] == ',' or index == len(row)-1:
                    string_builder.append(row[start:index])  # list builder
                    start = index + 1
            actor_grade_list.append(string_builder)  # add list to outer list
    infile.close()

# ***********************************************************************
# ENTRY POINT TO PROGRAM
# define an empty list with scope across the functions
actor_grade_list = []

# read in data for processing within the menu
read_file()
```

ts Reserved.

List Example 4 (3)

Optimised file reading

```
def read_file():
    infile = open("actor_grades.txt")

    # optimised method using .split()
    for row in infile:
        if not row.startswith("#"):  # ignore the header comment in the data file
            row = row.rstrip('\n').split(', ') # strip & split
            actor_grade_list.append(row)  # each element now contains a 'row'
    infile.close()


# ******************************************************************
# ENTRY POINT TO PROGRAM
# define an empty list with scope across the functions
actor_grade_list = []

# read in data for processing within the menu
read_file()
```

Pearson Copyright © 2022 Pearson Education, Ltd. All Rights Reserved.

List Example 4 (4)

Processing Lists

```
def print_summary():
    total_score = 0
    for item in range(len(actor_grade_list)):
        print(actor_grade_list[item][0],actor_grade_list[item][1], \
            actor_grade_list[item][2],sep=' -> ')
        total_score += float(actor_grade_list[item][2])

    print('\nCombined Scores:',format(total_score,'.2f'))
    print('Average Score:',format(total_score/len(actor_grade_list),'.2f'))


def minimum(act_g_lst):
    minimum = act_g_lst[0]

    for item in act_g_lst:
        if item[2] < minimum[2]:
            minimum = item
    return minimum




print_summary()
print(minimum(actor_grade_list))
```

```
B002 ->  Daniel Craig ->  30
B007 ->  Dr Who ->  70.9
B008 ->  Jonny English ->  60.2
B009 ->  Ethan Hunt ->  20

Combined Scores: 181.10
Average Score: 45.28

['B009', ' Ethan Hunt', ' 20']
```

ights Reserved.

Side note: Lambda functions I

anonymous functions introduction

- The lambda keyword in Python provides a shortcut for declaring small anonymous functions.
  - Lambda functions behave just like regular functions declared with the def keyword.

```
#function to add
def add(x, y):
    return x + y
print(add(4,5))

#equivalent to this lambda
print((lambda x, y: x + y)(4,5))
```

Parameters

Processing

Inputs

Pearson Copyright © 2022 Pearson Education, Ltd. All Rights Reserved.

Side note: Lambda functions II

Anonymous functions for sorting

- A frequent use case for lambdas is writing short and concise *key functions* for sorting iterables by an alternative key e.g. a nested listed.

```
actor_grade_list = ['B009', 'Ethan Hunt', '20']
                   ['B001', 'James Bond', '45']
                   ['B008', 'Jonny English', '60.2']
```

- If we call min() – how will this list be sorted?
- How do we specify that this be sorted by grade?

```
# Write our own custom function
def minimum(act_g_lst):
    minimum = act_g_lst[0]

    for item in act_g_lst:
        if item[2] < minimum[2]:
            minimum = item
    return minimum

print(minimum(actor_grade_list))

# OR, employ a lambda function
print(min(actor_grade_list, key=lambda x: x[2]))
```

Sequence objects may be compared to other objects with the same sequence type.

The comparison uses **lexicographical** ordering.

Pearson Copyright © 2022 Pearson Education, Ltd. All Rights Reserved.

Side Note: List Comprehensions

Concise way to create lists

- `vals = [expression for value in collection if condition]`
- This is equivalent to:

```
vals = [] for value in collection:
    if condition:
    vals.append(expression)
```

- Example:

```
>>> my_list = [x for x in range(10)]
>>> print(my_list)
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

>>> even_squares = [x * x for x in range(10) if not x % 2]
>>> print(my_list)
[0, 4, 16, 34, 64]
```

Ulster University

Pearson Copyright © 2022 Pearson Education, Ltd. All Rights Reserved.

Try it out

Exercise 3

Extend the actor grades file and program as follows:

1. Alter the data file, separating name into first name and surname, and adding in 'year of study' as an additional element.
2. Update the print function to accommodate the changes in step 1.
3. Add a method to prompt the user for input of a 'surname' prior to searching and returning relevant list items should a match be located within the list.

Ulster University

Pearson Copyright © 2022 Pearson Education, Ltd. All Rights Reserved.

Ulster University

Section 2

Working with Tuples

Tuples

A very brief overview

- **Tuple**: an **immutable** sequence
  - Very similar to a list
  - …but, once it is created it cannot be changed
  - Format: `tuple_name = (item1, item2)`
  - Tuples support a number of same operations as lists
    - Subscript indexing for retrieving elements
    - Methods such as `index`
    - Built in functions such as `len, min, max`
    - Slicing expressions
    - The `in`, `+`, and `*` operators

Ulster University

Pearson Copyright © 2022 Pearson Education, Ltd. All Rights Reserved.

Tuples (cont'd.)

- Tuples **do not** support the following 'manipulative' methods:
  - `append`
  - `remove`
  - `insert`
  - `reverse`
  - `sort`

Ulster University

Pearson Copyright © 2022 Pearson Education, Ltd. All Rights Reserved.

Tuples (cont'd.)

- Advantages for using tuples over lists:
  - Processing tuples is **faster** than processing lists
  - Tuples are **safer** options for static datasets
  - Methods exist to **convert** lists and tuples
    - *`list()` function: converts tuple to list*
    - *`tuple()` function: converts list to tuple*

Ulster University

Pearson Copyright © 2022 Pearson Education, Ltd. All Rights Reserved.

Quiz

Answer the following

1. What are the main differences between Lists and Tuples?

2. Give two reasons why Tuples exist.

3. Assume my_list references a list – write the statement that converts this to a tuple

Ulster University

Pearson Copyright © 2022 Pearson Education, Ltd. All Rights Reserved.

Summary

- This lecture + tutorial covered:
  - Lists, including:
    - Repetition and concatenation operators
    - Indexing
    - Techniques for processing lists
    - Slicing and copying lists
    - List methods and built-in functions for lists
    - Two-dimensional lists
  - Tuples, including:
    - Immutability
    - Difference from and advantages over lists

Ulster University

Pearson Copyright © 2022 Pearson Education, Ltd. All Rights Reserved.